Pata Hogar Desktop - Documentación técnica

Aplicación de escritorio de administración del sistema Pata Hogar

Dato	Valor
Versión del documento	1.0
Fecha	Octubre de 2026
Plataforma	.NET 10 (`net10.0-windows`), Windows Forms
Base de datos	MySQL 8.x (compartida con el sistema web)
Autores	Sebastián Carbonetti, Santiago Monterubianessi

---

1. Introducción

1.1 Propósito

Este documento describe el diseño y la implementación de **Pata Hogar Desktop**: su arquitectura, los componentes que la forman, el modelo de datos con el que trabaja, los flujos de ejecución, la configuración, los criterios de seguridad y las pautas para extenderla. Está destinado a quienes deban **mantener, auditar o ampliar** el sistema.

1.2 Alcance

La documentación abarca únicamente la aplicación de escritorio. El sitio web, su backend y la aplicación móvil de Pata Hogar son componentes independientes; solo se describen en la medida en que interactúan con esta aplicación (sección 2.3).

1.3 Documentos relacionados

Documento	Contenido
`README.md`	Presentación, requisitos, instalación, configuración y uso
`base_de_datos/crear_base_y_usuario.sql`	Script SQL de creación de la base de datos y del usuario

1.4 Terminología

Término	Definición
Solicitud de verificación	Pedido de una organización (refugio, veterinaria o asociación) para obtener una cuenta verificada en el sitio web
Modelo, Vista, Controlador	Las tres capas del patrón MVC (ver sección 4)
Vista pasiva	Variante de MVC en la que la vista no contiene lógica: solo presenta datos y notifica las acciones del usuario
Contrato de vista	Interfaz que enumera todo lo que un controlador puede leer, escuchar y ordenar a una vista

Término	Definición
Repositorio	Clase que encapsula las consultas a la base de datos y ofrece operaciones de alto nivel
Modo de respaldo	Comportamiento ante la falta de conexión: se muestran datos de ejemplo en lugar de datos reales

---

2. Contexto del sistema

2.1 Componentes de Pata Hogar

Pata Hogar es una plataforma de adopción de mascotas que funciona como un marketplace: usuarios que tienen mascotas para dar en adopción las publican, y otros usuarios las consultan y se contactan con quien publicó. Está compuesta por:

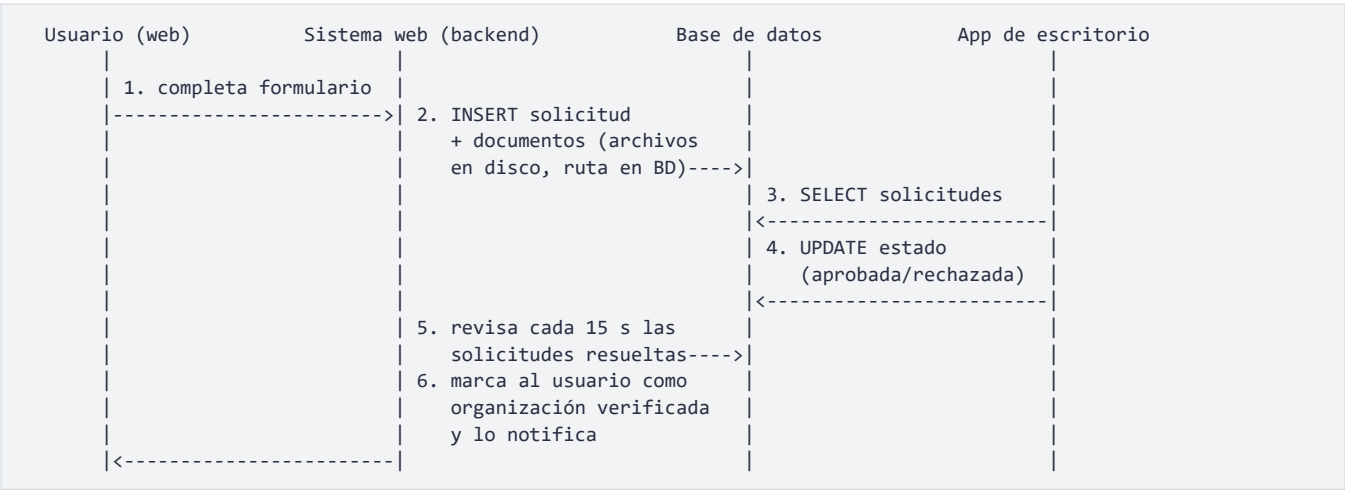
Componente	Tecnología	Función
Sitio web (frontend)	React + Vite	Interfaz pública para publicar, buscar y contactar
Backend web	Node.js + Express + Knex	API, autenticación, chat, notificaciones, **migraciones de la base de datos**
Base de datos	MySQL 8.x	Almacenamiento compartido por todos los componentes
Aplicación de escritorio	**C# / Windows Forms**	**Administración interna (este documento)**

2.2 Rol de la aplicación de escritorio

La aplicación de escritorio es una herramienta **interna del administrador**. No está pensada para el público general: el usuario final utiliza el sitio web. Su función actual es la **revisión de solicitudes de verificación**: un administrador examina la documentación adjunta por una organización y decide si aprobar o rechazar su solicitud.

2.3 Integración con el sistema web

La aplicación y el sistema web **no se comunican entre sí directamente**: se integran a través de la base de datos y del sistema de archivos.



Contrato de datos entre ambos sistemas

Responsable	Acción	Tabla / columnas involucradas
Sistema web	Crea la solicitud con estado `pendiente`	`verification_requests` (todas las columnas de datos)
Sistema web	Guarda cada archivo en disco y registra su ruta absoluta	`verification_documents.file_path`, `original_filename`
Aplicación de escritorio	Lee solicitudes y documentos	`verification_requests`, `verification_documents`
Aplicación de escritorio	Resuelve la solicitud	`verification_requests.status`, `rejection_reason`, `resolved_at`
Sistema web	Detecta las solicitudes resueltas que no notificó (`user_notified_at` nulo), asigna la insignia (`users.is_verified_organization`) y envía la notificación	`verification_requests.user_notified_at`, `users`, `notifications`

Consecuencias de este diseño:

- La aplicación de escritorio **no invoca la API web**: depende únicamente de la base de datos y de los archivos adjuntos.
- Los archivos adjuntos se referencian por **ruta absoluta del sistema de archivos**. La aplicación debe ejecutarse en un equipo con acceso a esa ruta (habitualmente, el mismo en el que corre el backend web).
- El backend web procesa las resoluciones con un intervalo de 15 segundos; la insignia y la notificación **no son inmediatas** y requieren que el backend esté en ejecución (si estuvo apagado, procesa las pendientes al iniciar).

---

3. Requisitos

3.1 Requisitos funcionales

Código	Requisito	Estado
RF-01	Autenticar a un administrador mediante usuario y contraseña	Implementado
RF-02	Mostrar una ventana principal con menú de navegación y estado de la conexión a la base de datos	Implementado
RF-03	Listar las solicitudes de verificación almacenadas en la base de datos	Implementado
RF-04	Filtrar las solicitudes por estado (todas, pendientes, aprobadas, rechazadas)	Implementado
RF-05	Mostrar el detalle completo de una solicitud seleccionada	Implementado
RF-06	Abrir cada documento adjunto con la aplicación asociada del sistema operativo	Implementado
RF-07	Aprobar una solicitud pendiente con confirmación previa	Implementado
RF-08	Rechazar una solicitud pendiente exigiendo un motivo y con confirmación previa	Implementado
RF-09	Persistir en la base de datos el resultado, el motivo y la fecha de resolución	Implementado
RF-10	Cerrar la sesión y volver a la pantalla de autenticación	Implementado
RF-11	Continuar operando, con datos de respaldo, si la base de datos no responde	Implementado
RF-12	Moderar reportes de publicaciones	Planificado

3.2 Requisitos no funcionales

Código	Requisito	Cumplimiento
RNF-01	Mantenibilidad: separación de responsabilidades	Arquitectura MVC con vistas pasivas (sección 4)
RNF-02	Verificabilidad: lógica comprobable sin interfaz gráfica	Los controladores dependen solo de interfaces (sección 12)
RNF-03	Seguridad: sin credenciales ni datos personales en el repositorio	Configuración local excluida del control de versiones (sección 10)
RNF-04	Seguridad: consultas inmunes a inyección SQL	Todas las consultas usan parámetros
RNF-05	Robustez: un fallo de conexión no debe cerrar la aplicación	Captura de excepciones y modo de respaldo (sección 9)
RNF-06	Usabilidad: aspecto coherente y minimalista	Paleta y controles centralizados (carpeta `Estilos/`)
RNF-07	Legibilidad: código autoexplicativo	Nombres propios en español y estructuras simples

3.3 Restricciones

- Plataforma exclusiva de **Windows** (Windows Forms).
- Requiere el **sistema web instalado** (genera la estructura de la base y guarda los archivos).
- La conexión a la base de datos es **síncrona**.

---

4. Arquitectura

4.1 Estilo arquitectónico

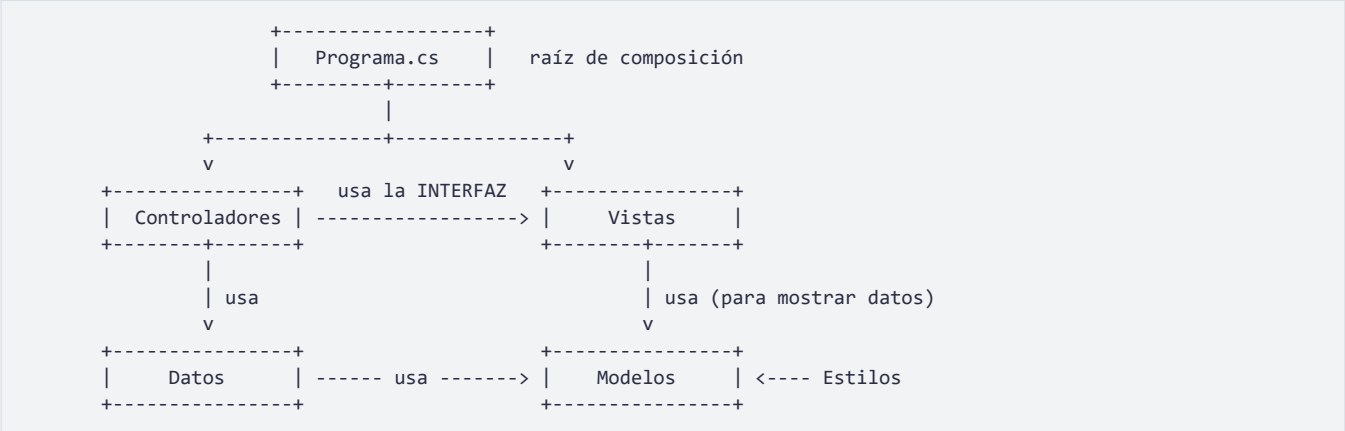
La aplicación adopta el patrón **MVC (Modelo - Vista - Controlador)** en su variante de **vista pasiva** (*Passive View*):

- La **Vista** no contiene lógica. Presenta datos y, ante una acción del usuario, guarda el dato relevante en una propiedad y dispara un evento.
- El **Controlador** se suscribe a esos eventos, consulta o modifica el Modelo y ordena a la Vista qué mostrar. Conoce a la Vista **únicamente a través de una interfaz**.
- El **Modelo** representa los datos y las reglas, y encapsula el acceso a la base de datos. No conoce a la Vista ni al Controlador.

4.2 Capas y responsabilidades

Capa	Carpetas	Responsabilidad
Modelo	`Modelos/`	Entidades de dominio, enumeraciones, reglas de negocio (autenticación) y configuración local
Modelo (datos)	`Datos/`	Conexión a MySQL y consultas
Vista	`Vistas/`	Pantallas, tarjetas y contratos de vista (interfaces)
Vista (estilo)	`Estilos/`	Paleta de colores, fuentes y controles visuales reutilizables
Controlador	`Controladores/`	Lógica de cada pantalla: estado, reglas de presentación, orquestación
Raíz de composición	`Programa.cs`	Crea las vistas y los controladores y gestiona el ciclo de sesión

4.3 Reglas de dependencia



Regla	Descripción
D-1	`Modelos` no depende de ninguna otra carpeta del proyecto
D-2	`Datos` depende únicamente de `Modelos`
D-3	`Vistas` depende de `Modelos` y `Estilos`; **nunca** de `Datos` ni de `Controladores`
D-4	`Controladores` depende de `Modelos`, `Datos` y de las **interfaces** de `Vistas`; no utiliza ningún tipo de Windows Forms
D-5	Solo `Programa.cs` conoce todas las capas

Estas reglas se verifican revisando los espacios de nombres importados por cada carpeta. Excepción documentada: `ControladorPrincipal` instancia la vista concreta `VistaSolicitudesVerificacion` al abrir esa sección (sección 15, L-8).

4.4 Mecanismos de comunicación

Sentido	Mecanismo	Ejemplos
Vista → Controlador (avisos)	Eventos `EventHandler`	`IngresarSolicitado`, `AprobarSolicitado`
Vista → Controlador (datos)	Propiedades de solo lectura del contrato	`Usuario`, `SolicitudElegida`, `MotivoRechazoEscrito`
Controlador → Vista (órdenes)	Métodos del contrato	`MostrarError`, `MostrarListado`, `MostrarDetalle`

El evento informa **qué ocurrió**; la propiedad informa **sobre qué dato**. La vista asigna la propiedad antes de disparar el evento.

4.5 Patrones de diseño aplicados

Patrón	Dónde se aplica	Propósito
MVC (vista pasiva)	Toda la aplicación	Separar presentación, lógica y datos
Observador (eventos)	Vistas y controladores	Desacoplar el aviso de la acción de su tratamiento
Repositorio	`RepositorioSolicitudes`	Aislar el SQL del resto del sistema
Raíz de composición	`Programa.cs`	Centralizar la creación y conexión de objetos
Plantilla de tema	`TemaApp`	Centralizar colores y fuentes

Patrón	Dónde se aplica	Propósito
Herencia de controles	`PanelRedondeado`, `BotonRedondeado`, `BotonNavegacion`	Extender controles estándar con dibujo propio

4.6 Decisiones de diseño

Decisión	Justificación
Vista pasiva con interfaces	Permite verificar los controladores con vistas simuladas, sin abrir ventanas
Recarga de datos desde la base tras cada resolución	La pantalla refleja siempre el estado realmente persistido
Propiedad + evento para transmitir datos	Evita definir clases `EventArgs` para cada aviso y mantiene los contratos simples
Detalle de solicitud construido por código	El contenido varía por solicitud (cantidad de documentos, motivo de rechazo); un diseño estático no es adecuado
Posicionamiento con cursor vertical (`_proximoY`)	Reemplaza cálculos de posición por un único valor acumulado
Controles con dibujo propio (GDI+)	Permiten esquinas redondeadas, que Windows Forms no ofrece de forma nativa
Nullabilidad desactivada (`<Nullable>disable`)	Evita anotaciones `?` y simplifica la lectura del código
Modo de respaldo ante falla de conexión	La aplicación no se interrumpe y el estado de la conexión es visible
Nombres propios en español	Coherencia con el dominio y el equipo; solo se conservan en inglés los nombres de .NET/Windows Forms y las columnas de la base compartida

4.7 Ciclo de vida de la aplicación

`Programa.Main` ejecuta un bucle que repite el ciclo de sesión mientras el administrador solicite cerrar sesión:

1. Se crea `VistaInicioSesion` y `ControladorInicioSesion`, y se muestra el diálogo modal.
2. Si el resultado no es `DialogResult.OK`, la aplicación termina.
3. Se crea `VistaPrincipal` y `ControladorPrincipal`, y se inicia el ciclo de mensajes de Windows con `Application.Run`.
4. Al cerrarse la ventana principal, si `ControladorPrincipal.CerrarSesionSolicitada` es verdadero se repite el ciclo desde el paso 1; en caso contrario, la aplicación termina.

---

5. Estructura del repositorio y catálogo de componentes

5.1 Árbol de directorios

```
/
|-- README.md
|-- DOCUMENTACION_TECNICA.md / .pdf
|-- .gitignore
|-- SistemaAdministracionPataHogar.slnx          solución de Visual Studio
|-- base_de_datos/
|   `-- crear_base_y_usuario.sql                 creación de la base y del usuario
`-- SistemaAdministracionPataHogar/              proyecto
    |-- SistemaAdministracionPataHogar.csproj
    |-- configuracion.ejemplo.json               plantilla de configuración
    |-- Programa.cs
    |-- Modelos/
    |   |-- SolicitudVerificacion.cs
    |   |-- DocumentoAdjunto.cs
    |   |-- TipoOrganizacion.cs
    |   |-- EstadoSolicitud.cs
    |   |-- ServicioAutenticacion.cs
    |   `-- ConfiguracionLocal.cs
    |-- Datos/
    |   |-- ConexionBaseDatos.cs
    |   `-- RepositorioSolicitudes.cs
    |-- Vistas/
    |   |-- IVistaInicioSesion.cs
    |   |-- IVistaPrincipal.cs
    |   |-- IVistaSolicitudesVerificacion.cs
    |   |-- VistaInicioSesion.cs / .Designer.cs / .resx
    |   |-- VistaPrincipal.cs / .Designer.cs / .resx
    |   |-- VistaSolicitudesVerificacion.cs / .Designer.cs
    |   `-- VistaTarjetaSolicitud.cs / .Designer.cs
    |-- Controladores/
    |   |-- ControladorInicioSesion.cs
    |   |-- ControladorPrincipal.cs
    |   `-- ControladorSolicitudesVerificacion.cs
    `-- Estilos/
        |-- TemaApp.cs
        |-- PanelRedondeado.cs
        |-- BotonRedondeado.cs
        `-- BotonNavegacion.cs
```

5.2 Catálogo de componentes

Componente	Capa	Responsabilidad
`Programa`	Raíz	Punto de entrada; crea vistas y controladores; ciclo de sesión
`SolicitudVerificacion`	Modelo	Entidad con los datos de una solicitud
`DocumentoAdjunto`	Modelo	Entidad con nombre y ruta de un archivo adjunto
`TipoOrganizacion` / `TipoOrganizacionAyudante`	Modelo	Enumeración de tipos y funciones de texto e icono
`EstadoSolicitud` / `EstadoSolicitudAyudante`	Modelo	Enumeración de estados y función de texto
`ServicioAutenticacion`	Modelo	Regla de validación de credenciales de administrador
`ConfiguracionLocal`	Modelo	Lectura y validación del archivo de configuración local
`ConexionBaseDatos`	Datos	Cadena de conexión, creación y prueba de conexiones MySQL
`RepositorioSolicitudes`	Datos	Consultas de lectura y actualización de solicitudes

Componente	Capa	Responsabilidad
`IVistaInicioSesion`, `IVistaPrincipal`, `IVistaSolicitudesVerificacion`	Vista	Contratos de cada pantalla
`VistaInicioSesion`	Vista	Ventana de autenticación
`VistaPrincipal`	Vista	Ventana principal: menú lateral y área de contenido
`VistaSolicitudesVerificacion`	Vista	Pantalla de lista, filtros y detalle de solicitudes
`VistaTarjetaSolicitud`	Vista	Tarjeta que representa una solicitud en la lista
`ControladorInicioSesion`	Controlador	Validación del acceso
`ControladorPrincipal`	Controlador	Navegación, estado de la base y cierre de sesión
`ControladorSolicitudesVerificacion`	Controlador	Filtrado, selección, apertura de documentos, aprobación y rechazo
`TemaApp`	Estilo	Colores y fuentes de la aplicación
`PanelRedondeado`, `BotonRedondeado`, `BotonNavegacion`	Estilo	Controles con dibujo propio

---

6. Modelo de datos

La estructura de la base de datos es **propiedad del sistema web**: la crean sus migraciones. La aplicación de escritorio no crea ni modifica tablas; utiliza las dos siguientes.

6.1 Tabla `verification_requests`

Una fila por solicitud de verificación.

Columna	Tipo	Nulable	Uso en la aplicación
`id`	int unsigned, clave primaria	No	Lectura; clave para actualizar
`user_id`	int unsigned, FK a `users.id`	No	No se utiliza
`organization_name`	varchar(150)	No	Lectura
`organization_type`	enum(`refugio`, `veterinaria`, `asociacion`)	No	Lectura
`responsible_name`	varchar(150)	No	Lectura
`email`	varchar(190)	No	Lectura
`phone`	varchar(30)	No	Lectura
`address`	varchar(255)	No	Lectura
`city`, `province`	varchar(120)	No	Lectura
`years_in_operation`	int unsigned	No	Lectura
`animals_housed`	int unsigned	No	Lectura
`website`	varchar(255)	Sí	Lectura
`description`	text	No	Lectura
`status`	enum(`pendiente`, `aprobada`, `rechazada`), valor inicial `pendiente`	No	**Lectura y escritura**

Columna	Tipo	Nulable	Uso en la aplicación
`rejection_reason`	text	Sí	**Lectura y escritura**
`resolved_at`	timestamp	Sí	**Lectura y escritura**
`resolved_by`	int unsigned, FK a `users.id`	Sí	No se utiliza (queda nulo)
`created_at`, `updated_at`	timestamp	No	`created_at`: lectura
`user_notified_at`	timestamp	Sí	No se utiliza; lo gestiona el sistema web

6.2 Tabla `verification_documents`

Una fila por archivo adjunto. Una solicitud puede tener varias filas, incluso del mismo tipo de documento.

Columna	Tipo	Nulable	Uso en la aplicación
`id`	int unsigned, clave primaria	No	Ordenamiento
`request_id`	int unsigned, FK a `verification_requests.id` (`ON DELETE CASCADE`)	No	Filtro
`document_type`	enum(`estatuto`, `dni_responsable`, `habilitacion_municipal`, `fotos_instalaciones`)	No	No se utiliza
`original_filename`	varchar(255)	No	Nombre visible del documento
`file_path`	varchar(500)	No	Ruta absoluta del archivo, usada para abrirlo
`created_at`, `updated_at`	timestamp	No	No se utilizan

El archivo en sí **no se almacena en la base de datos**: solo su ruta.

6.3 Correspondencia entre columnas y propiedades

Columna	Propiedad de `SolicitudVerificacion`
`id`	`Id`
`organization_name`	`NombreOrganizacion`
`organization_type`	`Tipo`
`responsible_name`	`NombreResponsable`
`email`	`Correo`
`phone`	`Telefono`
`address`	`Direccion`
`city`	`Ciudad`
`province`	`Provincia`
`years_in_operation`	`AniosFuncionamiento`
`animals_housed`	`AnimalesAlbergados`
`website`	`SitioWebORedes`

Columna	Propiedad de `SolicitudVerificacion`
`description`	`Descripcion`
`status`	`Estado`
`rejection_reason`	`MotivoRechazo`
`resolved_at`	`FechaResolucion`
`created_at`	`FechaSolicitud`
`verification_documents.original_filename`	`DocumentoAdjunto.Nombre`
`verification_documents.file_path`	`DocumentoAdjunto.RutaArchivo`

6.4 Correspondencia entre enumeraciones

Los valores de las columnas `enum` coinciden con los nombres de las enumeraciones de C#, salvo por mayúsculas y minúsculas. La traducción la realizan `ConvertirTipoOrganizacion` y `ConvertirEstadoSolicitud` en `RepositorioSolicitudes`.

Valor en la base	Enumeración
`refugio`	`TipoOrganizacion.Refugio`
`veterinaria`	`TipoOrganizacion.Veterinaria`
`asociacion`	`TipoOrganizacion.Asociacion`
`pendiente`	`EstadoSolicitud.Pendiente`
`aprobada`	`EstadoSolicitud.Aprobada`
`rechazada`	`EstadoSolicitud.Rechazada`

Un valor desconocido se interpreta como `Asociacion` (tipo) o `Pendiente` (estado).

6.5 Ciclo de vida del estado

```
                +--> aprobada   (status = 'aprobada', rejection_reason = NULL, resolved_at = NOW())
pendiente -----+
                +--> rechazada  (status = 'rechazada', rejection_reason = <motivo>, resolved_at = NOW())
```

- La aplicación solo ofrece resolver solicitudes en estado `pendiente`; las resueltas se muestran en modo de solo lectura.
- La aplicación **no revierte** resoluciones.
- La restricción a solicitudes pendientes se aplica en la interfaz; las sentencias `UPDATE` no verifican el estado previo (ver L-6 en la sección 15).

6.6 Privilegios recomendados para el usuario de la base de datos

El script de creación otorga todos los privilegios sobre la base para simplificar la instalación conjunta con el sistema web. Para un usuario dedicado exclusivamente a esta aplicación, bastan los siguientes:

Tabla	Privilegios mínimos
`verification_requests`	`SELECT`, `UPDATE` (`status`, `rejection_reason`, `resolved_at`)

Tabla	Privilegios mínimos
`verification_documents`	`SELECT`

---

7. Especificación de componentes

7.1 `Programa` (raíz de composición)

Punto de entrada (`Main`, con el atributo `[STAThread]`, requerido por Windows Forms).

Responsabilidad	Detalle
Inicialización	`ApplicationConfiguration.Initialize()`: estilos visuales, DPI y fuente por defecto
Configuración	Carga `ConfiguracionLocal`. Si el archivo falta o es inválido, muestra un cuadro de diálogo con el motivo y termina
Ciclo de sesión	Bucle descrito en la sección 4.7
Composición	Crea `VistaInicioSesion` + `ControladorInicioSesion` y `VistaPrincipal` + `ControladorPrincipal`

El diálogo de inicio de sesión se muestra con `ShowDialog` (modal). La ventana principal se ejecuta con `Application.Run`.

7.2 Modelos (`Modelos/`)

`SolicitudVerificacion`

Entidad sin comportamiento.

Propiedad	Tipo	Descripción
`Id`	`int`	Identificador. Los valores negativos (-1, -2) identifican datos de respaldo
`NombreOrganizacion`	`string`	Nombre de la organización
`Tipo`	`TipoOrganizacion`	Refugio, veterinaria o asociación
`NombreResponsable`	`string`	Persona a cargo
`Correo`, `Telefono`	`string`	Datos de contacto
`Direccion`, `Ciudad`, `Provincia`	`string`	Ubicación
`FechaSolicitud`	`DateTime`	Fecha de creación
`AniosFuncionamiento`	`int`	Antigüedad en años
`AnimalesAlbergados`	`int`	Cantidad de animales (0 en veterinarias)
`Descripcion`	`string`	Descripción de la organización
`SitioWebORedes`	`string`	Opcional (puede ser `null`)
`Documentos`	`List<DocumentoAdjunto>`	Archivos adjuntos
`Estado`	`EstadoSolicitud`	Valor inicial `Pendiente`
`MotivoRechazo`	`string`	`null` si no fue rechazada

Propiedad	Tipo	Descripción
`FechaResolucion`	`DateTime?`	`null` si sigue pendiente

`DocumentoAdjunto`

Propiedad	Descripción
`Nombre`	Nombre visible (nombre original del archivo)
`RutaArchivo`	Ruta absoluta del archivo

`TipoOrganizacion` **y** `EstadoSolicitud`

Enumeraciones. Cada una se acompaña de una clase estática de funciones de presentación:

Clase	Funciones
`TipoOrganizacionAyudante`	`ObtenerTexto(TipoOrganizacion)`: texto visible. `ObtenerIcono(TipoOrganizacion)`: símbolo asociado
`EstadoSolicitudAyudante`	`ObtenerTexto(EstadoSolicitud)`: texto visible

`ServicioAutenticacion`

Miembro	Descripción
`CredencialesValidas(string usuario, string contrasena)`	Devuelve `true` si coinciden, con distinción de mayúsculas y minúsculas, con las credenciales de administrador definidas en `ConfiguracionLocal`

Es la única regla de autenticación del sistema. Si se reemplaza el origen de las credenciales (por ejemplo, una tabla de usuarios), solo esta clase requiere cambios.

`ConfiguracionLocal`

Lee y valida el archivo `configuracion.local.json` (ver sección 10.1).

Miembro	Descripción
`Cargar()`	Lee el archivo, valida que existan todas las claves requeridas y las almacena. Lanza `InvalidOperationException` con un mensaje que indica el archivo o la clave faltante. Las propiedades invocan `Cargar()` automáticamente si todavía no se cargó
`Servidor`, `Puerto`, `NombreBaseDeDatos`, `UsuarioBaseDeDatos`, `ContrasenaBaseDeDatos`	Datos de conexión a MySQL
`UsuarioAdministrador`, `ContrasenaAdministrador`	Credenciales de acceso a la aplicación

7.3 Datos (`Datos/`)

`ConexionBaseDatos`

Clase estática que centraliza la conexión a MySQL mediante **MySqlConnector** (API ADO.NET).

Método	Descripción
`ObtenerCadenaDeConexion()`	Construye la cadena de conexión a partir de `ConfiguracionLocal`
`CrearConexion()`	Devuelve un `MySqlConnection` nuevo **sin abrir**. El llamador debe abrirlo y liberarlo (`using`)
`ProbarConexion(out string mensaje)`	Intenta abrir y cerrar una conexión. Devuelve `true`/`false` y, en `mensaje`, una descripción. No propaga excepciones

Parámetros de la cadena de conexión:

Parámetro	Valor
`Server`, `Port`, `Database`, `Uid`, `Pwd`	Tomados de la configuración local
`Connection Timeout`	`5` segundos

`RepositorioSolicitudes`

Única clase que ejecuta SQL sobre solicitudes. Todas las consultas están parametrizadas.

Método público	Descripción	Comportamiento ante error
`ObtenerTodas()`	Devuelve las solicitudes con sus documentos, de la más reciente a la más antigua	Captura toda excepción y devuelve el conjunto de respaldo
`AprobarSolicitud(int idSolicitud)`	Marca la solicitud como aprobada	Propaga la excepción
`RechazarSolicitud(int idSolicitud, string motivo)`	Marca la solicitud como rechazada con el motivo	Propaga la excepción

Lectura (`ObtenerTodasDesdeLaBaseDeDatos`). Una consulta recupera las solicitudes; luego, por cada una, se ejecuta una consulta de sus documentos. Los lectores de datos se cierran antes de ejecutar la siguiente consulta sobre la misma conexión.

```
SELECT id, organization_name, organization_type, responsible_name, email, phone,
       address, city, province, years_in_operation, animals_housed, website,
       description, status, rejection_reason, resolved_at, created_at
FROM verification_requests
ORDER BY created_at DESC;

SELECT original_filename, file_path
FROM verification_documents
WHERE request_id = @idSolicitud
ORDER BY id;
```

Los valores se convierten con `Convert.ToInt32`, `Convert.ToString` y `Convert.ToDateTime`, que son independientes del tipo exacto que entrega el proveedor (las columnas `int unsigned` no se corresponden con `int`). Las columnas nulables (`website`, `rejection_reason`, `resolved_at`) se comparan con `DBNull.Value` antes de convertirse.

Escritura.

```
-- AprobarSolicitud
UPDATE verification_requests
SET status = 'aprobada', rejection_reason = NULL, resolved_at = NOW()
WHERE id = @idSolicitud;

-- RechazarSolicitud
UPDATE verification_requests
SET status = 'rechazada', rejection_reason = @motivo, resolved_at = NOW()
WHERE id = @idSolicitud;
```

`NOW()` es la fecha y hora del servidor MySQL. La columna `resolved_by` no se completa.

Conjunto de respaldo. `CrearSolicitudesDeRespaldo` devuelve dos solicitudes ficticias (`Id` -1 y -2), identificadas con el sufijo "(ejemplo sin conexión)" en el nombre. Sus documentos son archivos de texto que se generan en una carpeta temporal del sistema (`PataHogar/DocumentosEjemplo`) únicamente para poder probar la apertura de archivos.

7.4 Vistas (`Vistas/`)

Contratos

`IVistaInicioSesion`

Miembro	Tipo	Descripción
`Usuario`, `Contrasena`	Propiedades (lectura)	Texto ingresado
`IngresarSolicitado`	Evento	El usuario solicitó ingresar
`MostrarError(string)`	Método	Muestra un mensaje de error
`LimpiarError()`	Método	Oculta el mensaje de error
`CerrarConExito()`	Método	Cierra el diálogo con `DialogResult.OK`

`IVistaPrincipal`

Miembro	Tipo	Descripción
`VistaMostrada`	Evento	La ventana terminó de mostrarse
`AbrirSolicitudesSolicitado`	Evento	Se eligió la sección de solicitudes
`CerrarSesionSolicitado`	Evento	Se solicitó cerrar sesión
`MostrarBienvenida()`	Método	Muestra el mensaje inicial
`MostrarContenido(Control, string, string)`	Método	Reemplaza el área de contenido y actualiza título y subtítulo
`ResaltarSeccionSolicitudes()`	Método	Marca como activa la opción del menú
`MostrarEstadoBaseDeDatos(bool, string)`	Método	Muestra el indicador de conexión
`ConfirmarCerrarSesion()`	Método (`bool`)	Solicita confirmación
`CerrarVentana()`	Método	Cierra la ventana

`IVistaSolicitudesVerificacion`

Miembro	Tipo	Descripción
`FiltroElegido`	Propiedad	`"Todas"`, `"Pendiente"`, `"Aprobada"` o `"Rechazada"`
`SolicitudElegida`	Propiedad	Solicitud de la tarjeta seleccionada
`RutaDocumentoElegido`	Propiedad	Ruta del documento cuyo botón se accionó
`MotivoRechazoEscrito`	Propiedad	Texto de la caja de motivo (vacío si no existe)
`FiltroCambiado`, `SolicitudSeleccionada`, `AbrirDocumentoSolicitado`, `AprobarSolicitado`, `RechazarSolicitado`	Eventos	Acciones del usuario
`MostrarListado(List<SolicitudVerificacion>, string, int)`	Método	Dibuja las tarjetas; el tercer parámetro es el id a marcar (0 = ninguno)
`MarcarSolicitudSeleccionada(int)`	Método	Resalta una tarjeta
`MostrarDetalle(SolicitudVerificacion)` / `MostrarDetalleVacio()`	Métodos	Panel de detalle
`MostrarInformacion`, `MostrarAdvertencia`, `MostrarError`	Métodos	Cuadros de diálogo
`Confirmar(string, string)`	Método (`bool`)	Cuadro Sí/No

`VistaInicioSesion`

`Form` modal de tamaño fijo (900 x 560). Panel izquierdo (380 px) con la identidad visual y panel derecho con el formulario (320 x 372). Asigna `AcceptButton` al botón de ingreso, de modo que la tecla Enter equivale a accionarlo. No valida credenciales.

`VistaPrincipal`

`Form` maximizado (mínimo 1000 x 620).

```
VistaPrincipal
|-- panelBarraLateral (Dock Left, 240 px)
|     |-- lblMarca
|     |-- btnSolicitudes (BotonNavegacion)
|     `-- panelPieBarraLateral (Dock Bottom, 104 px)
|           |-- lblUsuarioActivo
|           |-- lblEstadoBaseDeDatos
|           `-- lnkCerrarSesion
`-- panelPrincipal (Dock Fill)
      |-- panelEncabezado (Dock Top, 76 px): lblTituloSeccion, lblSubtituloSeccion
      `-- panelContenido (Dock Fill, relleno 32/24/32/24)
```

`MostrarContenido` vacía `panelContenido`, asigna `Dock = Fill` al control recibido y lo agrega. El encabezado dibuja una línea inferior en el evento `Paint`.

`VistaSolicitudesVerificacion`

`UserControl` de dos columnas:

```
VistaSolicitudesVerificacion
|-- panelListado (Dock Left, 380 px, relleno derecho 20)
|     |-- panelFiltros (Dock Top, 40 px) -> flujoFiltros -> 4 botones de filtro
|     `-- flujoLista (Dock Fill, vertical, con desplazamiento): tarjetas
`-- panelDetalle (PanelRedondeado, Dock Fill)
      `-- pnlDetalleContenido (Dock Fill, con desplazamiento): detalle construido por código
```

- **Filtros.** Los cuatro botones comparten el manejador `BotonDeFiltro_Click`, que lee el valor de su propiedad `Tag` (`"Todas"`, `"Pendiente"`, `"Aprobada"`, `"Rechazada"`).
- **Listado.** `MostrarListado` crea una `VistaTarjetaSolicitud` por solicitud recibida, sin decidir cuáles se muestran. Si la lista está vacía muestra un mensaje.
- **Documentos.** Cada botón *Abrir* guarda en su `Tag` la ruta del archivo; el manejador común copia ese valor a `RutaDocumentoElegido` y dispara el evento.

Construcción del detalle con cursor vertical. El campo `_proximoY` indica la ordenada del siguiente control. Cada método `Agregar...` ubica su control en `_proximoY` y lo incrementa en la altura que ocupa. `MostrarDetalle` reinicia el cursor y encadena las llamadas; al finalizar, `AutoScrollMinSize` se fija en `_proximoY + 24`.

Elemento	Método	Altura que avanza (px)
Nombre de la organización	`AgregarNombreOrganizacion`	34
Tipo y etiqueta de estado	`AgregarFilaTipoYEstado`	40
Campo (título + valor)	`AgregarCampo`	44
Título de sección	`AgregarTituloDeSeccion`	28
Descripción	`AgregarTextoLargo`	74
Documento	`AgregarDocumento`	38
Línea separadora	`AgregarLineaSeparadora`	20
Caja de motivo y botones (pendiente)	`AgregarSeccionAprobarRechazar`	22 + 82 + 44
Resultado (resuelta)	`AgregarSeccionYaResuelta`	56 (aprobada) u 82 (rechazada)

El ancho de contenido es la constante `AnchoContenido` (660 px).

`VistaTarjetaSolicitud`

`UserControl` de 336 x 96 px. Muestra nombre, tipo y ubicación, fecha relativa ("Hoy", "Ayer", "Hace N días") y una etiqueta de estado coloreada. Conecta el mismo manejador al evento `Click` de sus seis controles internos y dispara el evento propio `Seleccionada`. `MarcarSeleccion(bool)` alterna borde y fondo de resaltado. El diseño finaliza con `ResumeLayout(true)` para que el `Dock = Fill` del panel interno se aplique.

7.5 Controladores (`Controladores/`)

Los controladores reciben la vista por su interfaz en el constructor y se suscriben a sus eventos. Ninguno utiliza tipos de Windows Forms.

`ControladorInicioSesion`

Método	Comportamiento
`AlSolicitarIngreso`	Lee `Usuario` (aplicando `Trim`) y `Contrasena`; consulta `ServicioAutenticacion`. Si son válidas: `LimpiarError()` y `CerrarConExito()`. Si no: `MostrarError(...)`

`ControladorPrincipal`

Elemento	Comportamiento
`CerrarSesionSolicitada`	Propiedad que consulta `Programa` al cerrarse la ventana
Constructor	Se suscribe a los tres eventos y ordena `MostrarBienvenida()`
`AlMostrarseVista`	`ConexionBaseDatos.ProbarConexion` y `MostrarEstadoBaseDeDatos`
`AlSolicitarAbrirSolicitudes`	Crea `VistaSolicitudesVerificacion` y su controlador, invoca `Iniciar()`, ordena `ResaltarSeccionSolicitudes()` y `MostrarContenido(...)`
`AlSolicitarCerrarSesion`	Si `ConfirmarCerrarSesion()` es verdadero: marca `CerrarSesionSolicitada` y ordena `CerrarVentana()`

`ControladorSolicitudesVerificacion`

Estado interno:

Campo	Contenido
`_vista`	Vista (`IVistaSolicitudesVerificacion`)
`_filtroActual`	Filtro activo (inicia en `"Todas"`)
`_solicitudSeleccionada`	Solicitud elegida (o `null`)
`_todasLasSolicitudes`	Última lista leída de la base de datos

Métodos:

Método	Comportamiento
`Iniciar()`	`CargarListado()` y `MostrarDetalleVacio()`
`CargarListado()`	Obtiene todas las solicitudes, conserva las que cumplen `CoincideConFiltro`, determina el id seleccionado y ordena `MostrarListado(...)`
`AlCambiarFiltro`	Actualiza `_filtroActual` desde `FiltroElegido` y recarga
`AlSeleccionarSolicitud` / `SeleccionarSolicitud`	Guarda la selección, ordena marcar la tarjeta y mostrar el detalle
`AlSolicitarAbrirDocumento`	Ejecuta `Process.Start` con `UseShellExecute = true` sobre la ruta; ante error ordena `MostrarError`
`AlSolicitarAprobar`	Confirma; ejecuta `AprobarSolicitud`; si falla, informa y no continúa; si no, `TrasResolucion` y `MostrarInformacion`
`AlSolicitarRechazar`	Recorta el motivo; si está vacío, `MostrarAdvertencia` y finaliza; confirma; ejecuta `RechazarSolicitud`; luego igual que la aprobación
`TrasResolucion(id)`	Restablece el filtro a `"Todas"`, recarga desde la base, localiza la solicitud actualizada y la selecciona

La secuencia de la aprobación es: **confirmar → persistir → recargar → informar**.

7.6 Estilos (`Estilos/`)

`TemaApp`

Clase estática con la paleta y las fuentes. Las fuentes usan la familia `Segoe UI`.

Miembro	Color (RGB)	Uso
`Principal`, `PrincipalOscuro`, `PrincipalClaro`	(58, 125, 68), (43, 97, 51), (224, 238, 226)	Acento, resaltado y selección
`Fondo`, `FondoTarjeta`, `Borde`	(247, 248, 250), blanco, (226, 229, 233)	Superficies y bordes
`BarraLateral`, `BarraLateralResaltada`, `BarraLateralSeleccionada`, `BarraLateralTexto`	(31, 41, 55), (45, 55, 72), (58, 125, 68), (214, 219, 226)	Menú lateral
`TextoPrincipal`, `TextoSecundario`	(43, 45, 66), (108, 122, 137)	Texto
`Exito`, `ExitoClaro`	(46, 174, 113), (224, 245, 233)	Estado aprobada
`Peligro`, `PeligroClaro`	(230, 73, 79), (252, 228, 229)	Estado rechazada y errores
`Advertencia`, `AdvertenciaClaro`	(224, 160, 30), (252, 240, 214)	Estado pendiente

`ObtenerColorDeEstado` y `ObtenerColorDeFondoDeEstado` devuelven el color fuerte y el claro correspondientes a un `EstadoSolicitud`.

`PanelRedondeado`

Hereda de `Panel`. Propiedades: `RadioEsquinas` (10), `ColorBorde`, `GrosorBorde` (1). Activa `UserPaint`, `AllPaintingInWmPaint`, `ResizeRedraw` y `OptimizedDoubleBuffer`. En `OnPaint`: (1) activa el suavizado; (2) construye un `GraphicsPath` rectangular con cuatro arcos de 90° (ángulos iniciales 180, 270, 0 y 90); (3) asigna `Region` con ese contorno; (4) rellena y dibuja el borde.

`BotonRedondeado`

Hereda de `Button`. Propiedades: `RadioEsquinas` (8), `ColorNormal`, `ColorResaltado`. Mantiene un indicador `_mouseEncima` actualizado por `MouseEnter`/`MouseLeave` e invoca `Invalidate()`. El color de relleno es gris si el control está deshabilitado, `ColorResaltado` con el puntero encima y `ColorNormal` en otro caso. El texto se centra con `TextRenderer.DrawText`.

`BotonNavegacion`

Hereda de `Button`. Rectangular, con texto alineado a la izquierda. La propiedad `Seleccionado` pinta el fondo con `BarraLateralSeleccionada` y agrega una franja blanca de 4 px; con el puntero encima usa `BarraLateralResaltada`.

Las propiedades públicas de los tres controles llevan el atributo `[DesignerSerializationVisibility(Visible)]`, requerido por el analizador de .NET 10 para que el diseñador de Visual Studio persista sus valores.

---

8. Flujos de ejecución

En cada flujo se indica el componente y el método que interviene en cada paso.

8.1 Inicio de sesión

1. `Programa.Main` carga la configuración y crea `VistaInicioSesion` y `ControladorInicioSesion`; este último se suscribe a `IngresarSolicitado`.
2. `ShowDialog()` muestra la ventana modal.
3. El usuario completa los campos y acciona *Ingresar* (o presiona Enter).
4. `VistaInicioSesion.btnIngresar_Click` dispara `IngresarSolicitado`.
5. `ControladorInicioSesion.AlSolicitarIngreso` lee `Usuario` y `Contrasena`, y consulta `ServicioAutenticacion.CredencialesValidas`.
6. Si son válidas: `LimpiarError()` y `CerrarConExito()`; el diálogo devuelve `DialogResult.OK` y `Programa` continúa con la ventana principal. Si no: `MostrarError("Usuario o contraseña incorrectos.")` y la ventana permanece abierta.
7. Si el diálogo se cierra con otro resultado, `Programa` finaliza la aplicación.

8.2 Ventana principal y estado de la base de datos

1. `Programa` crea `VistaPrincipal` y `ControladorPrincipal`; el constructor del controlador se suscribe a los tres eventos y ordena `MostrarBienvenida()`.
2. `Application.Run` muestra la ventana.
3. Al terminar de mostrarse, `VistaPrincipal` dispara `VistaMostrada`.
4. `ControladorPrincipal.AlMostrarseVista` invoca `ConexionBaseDatos.ProbarConexion` y ordena `MostrarEstadoBaseDeDatos(conectado, mensaje)`.

8.3 Apertura de la sección de solicitudes

1. `VistaPrincipal.btnSolicitudes_Click` dispara `AbrirSolicitudesSolicitado`.
2. `ControladorPrincipal.AlSolicitarAbrirSolicitudes`:
    1. crea `VistaSolicitudesVerificacion`;
    2. crea `ControladorSolicitudesVerificacion` (se suscribe a los cinco eventos de la vista);
    3. invoca `Iniciar()`: `CargarListado()` (lee la base y ordena `MostrarListado`) y `MostrarDetalleVacio()`;
    4. ordena `ResaltarSeccionSolicitudes()` y `MostrarContenido(...)`.

8.4 Filtrado

1. `BotonDeFiltro_Click` lee el `Tag` del botón, lo guarda en `_filtroElegido` y dispara `FiltroCambiado`.
2. `AlCambiarFiltro` copia `FiltroElegido` a `_filtroActual` y ejecuta `CargarListado()`.
3. `CargarListado` vuelve a leer la base, conserva las solicitudes que cumplen el filtro y ordena `MostrarListado(filtradas, filtro, idSeleccionado)`.

8.5 Selección de una solicitud

1. `VistaTarjetaSolicitud` dispara `Seleccionada` al recibir un clic en cualquiera de sus controles.
2. `VistaSolicitudesVerificacion.Tarjeta_Seleccionada` guarda la solicitud en `_solicitudElegida` y dispara `SolicitudSeleccionada`.
3. `AlSeleccionarSolicitud` lee `SolicitudElegida` y ejecuta `SeleccionarSolicitud`, que ordena `MarcarSolicitudSeleccionada(id)` y `MostrarDetalle(solicitud)`.

8.6 Apertura de un documento

1. `BotonAbrirDocumento_Click` lee la ruta del `Tag`, la guarda en `_rutaDocumentoElegido` y dispara `AbrirDocumentoSolicitado`.
2. `AlSolicitarAbrirDocumento` ejecuta `Process.Start` con `UseShellExecute = true`; el sistema operativo abre el archivo con la aplicación asociada.
3. Ante una excepción (por ejemplo, archivo inexistente) ordena `MostrarError`.

8.7 Aprobación

```
Vista                      Controlador                    Modelo/Datos
  |  AprobarSolicitado          |                              |
  |---------------------------->|                              |
  |   Confirmar(...)            |                              |
  |<----------------------------|                              |
  |   true                      |                              |
  |---------------------------->|  AprobarSolicitud(id)        |
  |                             |----------------------------->| UPDATE ... 'aprobada'
  |                             |  ObtenerTodas() (TrasResolucion)  |
  |                             |----------------------------->| SELECT ...
  |   MostrarListado / MostrarDetalle                          |
  |<----------------------------|                              |
  |   MostrarInformacion        |                              |
  |<----------------------------|                              |
```

1. `AprobarSolicitado` llega a `AlSolicitarAprobar`; si no hay solicitud seleccionada, termina.
2. `Confirmar(...)`: si la respuesta es negativa, no se realiza ninguna acción.
3. `RepositorioSolicitudes.AprobarSolicitud(id)`. Ante una excepción, `MostrarError` y se interrumpe el flujo (no se informa un éxito falso).
4. `TrasResolucion(id)`: filtro `"Todas"`, recarga desde la base, localiza la solicitud actualizada y la selecciona. El detalle muestra el estado resuelto.
5. `MostrarInformacion("Solicitud aprobada", ...)`.
6. Fuera de la aplicación: el backend web detecta la resolución, asigna la insignia y notifica al usuario (sección 2.3).

8.8 Rechazo

1. `RechazarSolicitado` llega a `AlSolicitarRechazar`, que lee `MotivoRechazoEscrito` y aplica `Trim`.
2. Si el motivo está vacío: `MostrarAdvertencia` y termina, sin solicitar confirmación.
3. Si hay motivo: `Confirmar(...)`; luego `RechazarSolicitud(id, motivo)`; el resto es idéntico a los pasos 4 y 5 de la aprobación.

8.9 Cierre de sesión

1. `lnkCerrarSesion_LinkClicked` dispara `CerrarSesionSolicitado`.
2. `AlSolicitarCerrarSesion` ordena `ConfirmarCerrarSesion()`.
3. Si se confirma: `CerrarSesionSolicitada = true` y `CerrarVentana()`.
4. `Application.Run` finaliza; `Programa` repite el ciclo de sesión (sección 4.7).

---

9. Manejo de errores y degradación

Situación	Tratamiento	Resultado para el usuario
Archivo de configuración ausente o incompleto	`ConfiguracionLocal.Cargar` lanza `InvalidOperationException`; `Programa` la captura al iniciar	Cuadro de diálogo con el archivo o la clave faltante; la aplicación termina
Credenciales inválidas	`ControladorInicioSesion` ordena `MostrarError`	Mensaje de error; se puede reintentar
Base de datos inaccesible al iniciar	`ProbarConexion` captura la excepción y devuelve `false`	Indicador rojo en el menú lateral
Falla de lectura de solicitudes	`ObtenerTodas` captura toda excepción	Se muestran las solicitudes de respaldo
Falla al aprobar o rechazar	La excepción se propaga y el controlador la captura	Mensaje de error; el estado no cambia
Archivo adjunto inexistente o sin aplicación asociada	El controlador captura la excepción de `Process.Start`	Mensaje de error con el detalle técnico
Rechazo sin motivo	El controlador valida antes de consultar al modelo	Advertencia; no se guarda nada
Servidor MySQL sin respuesta	Tiempo de espera de conexión de 5 segundos	La interfaz queda bloqueada hasta cumplirse (ver L-4)

Principio general: **ninguna falla de infraestructura debe cerrar la aplicación**, con la única excepción de una configuración inválida, que impide iniciar.

---

10. Configuración y seguridad

10.1 Archivo de configuración local

Los datos sensibles no se almacenan en el código fuente: se leen del archivo `configuracion.local.json`.

Aspecto	Detalle
Formato	JSON, leído con `System.Text.Json`
Ubicación en desarrollo	Carpeta del proyecto (`SistemaAdministracionPataHogar/`)
Ubicación en ejecución	Directorio del ejecutable; el archivo se copia desde la carpeta del proyecto durante la compilación
Control de versiones	**Excluido** (`.gitignore`). Solo se versiona la plantilla `configuracion.ejemplo.json`
Validación	`ConfiguracionLocal.Cargar` verifica la existencia de todas las claves

Estructura:

```
{
  "baseDeDatos": {
    "servidor": "127.0.0.1",
    "puerto": "3306",
    "nombre": "nombre_de_la_base",
    "usuario": "usuario_de_la_base",
    "contrasena": "contrasena_de_la_base"
  },
  "administrador": {
    "usuario": "usuario_administrador",
    "contrasena": "contrasena_administrador"
  }
}
```

Clave	Descripción
`baseDeDatos.servidor`, `puerto`, `nombre`	Ubicación y nombre de la base de datos
`baseDeDatos.usuario`, `contrasena`	Credenciales de MySQL
`administrador.usuario`, `contrasena`	Credenciales de acceso a la aplicación

10.2 Información que no debe versionarse

Elemento	Motivo
`configuracion.local.json`	Contiene contraseñas
Archivos adjuntos de usuarios (documentos de identidad, fotografías, PDF)	Datos personales
Capturas de pantalla o volcados de la base con datos reales	Datos personales
`bin/`, `obj/`, `.vs/`, `*.user`	Archivos generados o específicos del equipo

El archivo `.gitignore` del repositorio excluye estos elementos. Antes de cada publicación se recomienda revisar `git status`.

10.3 Medidas de seguridad aplicadas

- **Consultas parametrizadas.** Ningún valor se concatena en el texto SQL; se transmiten con `Parameters.AddWithValue`, lo que previene la inyección SQL.
- **Sin credenciales en el código fuente** ni en el repositorio.
- **Tiempo de espera de conexión** acotado (5 segundos).
- **Liberación de recursos** (`using`) en conexiones y lectores de datos.
- **Aislamiento de la lógica de autenticación** en una única clase.
- **Mensajes de error al usuario** que no exponen la cadena de conexión.

10.4 Riesgos y recomendaciones

Riesgo	Recomendación
Las contraseñas residen en texto plano en el archivo local	Aceptable en desarrollo. Para producción, utilizar un almacén de secretos o variables de entorno
Existe un único administrador, con credenciales de configuración	Migrar a una tabla de usuarios administradores con contraseñas almacenadas con *hash*

Riesgo	Recomendación
El usuario de la base tiene todos los privilegios sobre el esquema	Crear un usuario específico con los privilegios mínimos de la sección 6.6
Las rutas de los archivos adjuntos exponen la estructura del servidor	Mantener la aplicación en un equipo de confianza
Una contraseña publicada por error permanece en el historial de Git	Rotar la contraseña de inmediato; borrar el archivo no es suficiente

---

11. Compilación, ejecución y despliegue

11.1 Requisitos

- Windows 10 u 11.
- SDK de .NET 10.
- (Opcional) Visual Studio con la carga de trabajo *Desarrollo de escritorio con .NET*.
- Servidor MySQL 8.x con la base de datos del sistema web (ver `README.md`).

11.2 Compilación y ejecución

```
dotnet build SistemaAdministracionPataHogar.slnx
dotnet run --project SistemaAdministracionPataHogar
```

El paquete NuGet `MySqlConnector` (2.6.2) se restaura automáticamente.

11.3 Publicación

Para generar un ejecutable autónomo, que no requiere tener instalado el entorno de .NET:

```
dotnet publish SistemaAdministracionPataHogar -c Release -r win-x64 --self-contained
```

Junto al ejecutable generado debe colocarse el archivo `configuracion.local.json`. El servidor MySQL y el sistema web deben estar disponibles en el equipo de destino.

---

12. Verificación y pruebas

12.1 Estrategia

Las interfaces de vista permiten **verificar los controladores sin interfaz gráfica** mediante *vistas simuladas*: clases que implementan el contrato, no dibujan nada y registran las órdenes recibidas. Los eventos se disparan manualmente y se comprueba el resultado.

```
class VistaInicioSesionSimulada : IVistaInicioSesion
{
    public string Usuario { get; set; }
    public string Contrasena { get; set; }
    public event EventHandler IngresarSolicitado;
    public string ErrorMostrado;
    public bool CerroConExito;

    public void MostrarError(string mensaje) { ErrorMostrado = mensaje; }
    public void LimpiarError() { ErrorMostrado = null; }
    public void CerrarConExito() { CerroConExito = true; }
    public void Disparar() { IngresarSolicitado(this, EventArgs.Empty); }
}
```

12.2 Casos verificados

Componente	Caso	Resultado esperado
`ControladorInicioSesion`	Credenciales incorrectas	Se muestra el error; no se cierra
`ControladorInicioSesion`	Credenciales correctas con espacios en el usuario	Se cierra con éxito
`ControladorSolicitudesVerificacion`	Inicio	Se carga el listado y el detalle queda vacío
`ControladorSolicitudesVerificacion`	Filtro por estado	Solo se listan solicitudes de ese estado
`ControladorSolicitudesVerificacion`	Selección	Se muestra el detalle y se marca la tarjeta
`ControladorSolicitudesVerificacion`	Rechazo sin motivo	Advertencia; no se solicita confirmación
`ControladorSolicitudesVerificacion`	Aprobar o rechazar con confirmación negada	No se persiste ni se informa nada
`ControladorSolicitudesVerificacion`	Abrir un archivo inexistente	Mensaje de error
Interfaz	Renderizado de las ventanas con datos reales	Aspecto conforme al diseño
Arquitectura	Revisión de dependencias entre carpetas	Se cumplen las reglas D-1 a D-5

12.3 Lista de comprobación manual

- Iniciar sesión con credenciales válidas e inválidas.
- Verificar el indicador de conexión con la base encendida y apagada.
- Cargar una solicitud desde el sitio web y comprobar que aparece en la aplicación.
- Abrir cada tipo de documento adjunto.
- Aprobar una solicitud y verificar, en el sitio web, la insignia y la notificación.
- Rechazar una solicitud con motivo y verificar la notificación.
- Cerrar sesión y volver a ingresar.

Estado actual: las pruebas con vistas simuladas se ejecutan manualmente y **no están incorporadas al repositorio** (ver L-12).

13. Guía de extensión

13.1 Agregar un campo a la solicitud

Si la tabla incorpora una columna (por ejemplo, `cuit`):

1. **Modelo:** agregar la propiedad a `SolicitudVerificacion`.
2. **Datos:** incluir la columna en el `SELECT` de `RepositorioSolicitudes` y asignarla al leer; completarla también en el conjunto de respaldo.
3. **Vista:** agregar `AgregarCampo("CUIT", solicitud.Cuit);` en `MostrarDetalle`.

El controlador no requiere cambios.

13.2 Agregar una sección al menú

1. **Modelo y datos:** entidad en `Modelos/` y repositorio en `Datos/`.
2. **Contrato:** interfaz `IVistaXxx` en `Vistas/`.
3. **Vista:** `UserControl` que implemente la interfaz, sin lógica.
4. **Controlador:** clase en `Controladores/` que reciba la interfaz y use el repositorio.
5. **Menú:** botón en `VistaPrincipal`, evento en `IVistaPrincipal` y manejador en `ControladorPrincipal`, que crea la vista y el controlador y llama a `MostrarContenido`. Con más de una opción en el menú, generalizar `ResaltarSeccionSolicitudes` para desmarcar la opción anterior.

13.3 Cambiar el origen de la autenticación

Modificar únicamente `ServicioAutenticacion.CredencialesValidas` (por ejemplo, para consultar una tabla de usuarios con contraseñas almacenadas mediante *hash*). La vista y el controlador no se ven afectados.

13.4 Cambiar la identidad visual

Modificar los valores de `TemaApp`. Los controles propios y las vistas toman de allí los colores y las fuentes.

---

14. Convenciones de código

Aspecto	Convención
Idioma	Todos los nombres propios (clases, métodos, variables, archivos y carpetas) en español. Se conservan en inglés únicamente los nombres de .NET y Windows Forms y las columnas de la base de datos
Tipos y métodos	`PascalCase`
Campos privados	`_camelCase`
Variables locales y parámetros	`camelCase`
Controles	Prefijo por tipo: `btn`, `lbl`, `txt`, `pnl`, `panel`, `lnk`, `flujo`
Manejadores de controles	`control_Evento` (por ejemplo, `btnIngresar_Click`)
Manejadores de avisos de vista	Prefijo `Al` (por ejemplo, `AlSolicitarAprobar`)

Aspecto	Convención
Estilo	Estructuras explícitas (`foreach`, `if`); sin LINQ ni expresiones lambda como manejadores de eventos
Vistas	Sin lógica de negocio; no acceden a `Datos` ni a `Controladores`
Controladores	Sin tipos de Windows Forms; dependen solo de interfaces de vista
Consultas SQL	Siempre parametrizadas, y solo en `Datos/`
Datos sensibles	Nunca en el código ni en el repositorio

---

15. Limitaciones conocidas y evolución

15.1 Limitaciones

Código	Limitación	Posible mejora
L-1	Las contraseñas residen en texto plano en el archivo de configuración local	Almacén de secretos o variables de entorno
L-2	Autenticación con un único administrador de configuración	Tabla de usuarios con *hash* de contraseñas
L-3	No se registra qué administrador resolvió cada solicitud (`resolved_by` queda nulo)	Asociar la sesión a un usuario real
L-4	Las consultas son síncronas; con el servidor sin respuesta la interfaz se bloquea hasta 5 s	`async/await`
L-5	No se verifica el número de filas afectadas por los `UPDATE`	Comprobar el valor devuelto por `ExecuteNonQuery`
L-6	No se verifica el estado previo ni la concurrencia al resolver	`WHERE id = @id AND status = 'pendiente'` y control de filas afectadas
L-7	Consulta de documentos por solicitud (patrón N+1)	Una única consulta con `JOIN` o `IN`
L-8	`ControladorPrincipal` instancia la vista concreta de solicitudes	Fábrica de vistas o inyección de dependencias
L-9	Los controles reemplazados no se liberan con `Dispose`	Liberar al cambiar de sección
L-10	El detalle usa un ancho fijo de 660 px	Calcular el ancho según el panel
L-11	Los documentos se abren por ruta local	Requiere ejecutarse en el equipo que aloja los archivos
L-12	Las pruebas no están incorporadas al repositorio	Proyecto de pruebas versionado
L-13	Los iconos son caracteres emoji dependientes de la fuente del sistema	Imágenes o fuente de iconos
L-14	La insignia y la notificación son asíncronas y dependen de que el backend web esté en ejecución	Documentado; no requiere cambio en esta aplicación
L-15	El script de creación otorga todos los privilegios al usuario	Usuario con privilegios mínimos (sección 6.6)

15.2 Evolución prevista

- Moderación de reportes de publicaciones (RF-12).
- Ejecución asíncrona de las operaciones de base de datos.
- Proyecto de pruebas automatizadas.
- Autenticación basada en usuarios de la base de datos.
- Registro de auditoría de las resoluciones (`resolved_by`).

---

16. Glosario

Término	Definición
Contrato de vista	Interfaz que define lo que un controlador puede leer, escuchar y ordenar a una vista
Evento	Aviso emitido por un objeto al que otros pueden suscribirse
Designer	Archivo `.Designer.cs` con el armado visual de una pantalla (`InitializeComponent`)
`partial class`	Clase repartida en varios archivos
`Tag`	Propiedad de los controles de Windows Forms para asociar un dato arbitrario
Migración	Script versionado que crea o modifica la estructura de la base de datos (Knex en el sistema web)
Modo de respaldo	Presentación de datos de ejemplo cuando no hay conexión con la base
Raíz de composición	Punto único donde se crean y conectan los objetos de la aplicación
Vista simulada	Implementación de un contrato de vista sin interfaz gráfica, usada en pruebas

---

Apéndice A. Historial de versiones

Versión	Descripción
0.1	Inicio de sesión, menú principal y pantalla de solicitudes con datos de ejemplo
0.2	Simplificación del código e incorporación de la conexión a MySQL
0.3	Lectura y escritura reales de solicitudes y documentos
0.4	Reorganización de todo el sistema al patrón MVC con vistas pasivas
0.5	Renombrado de todos los elementos propios al español
1.0	Configuración local externa, script de base de datos, README y documentación técnica para publicación